

Лабораторная работа №1.1

Введение в MATLAB. Переменные, функции, графика

Цель работы: Изучение системы MATLAB для инженерных, финансовых и математических расчетов. Выполнение учебного математического расчета.

17	$f(x) = 1 - e^{2 \cdot x} $	$F(x, y) = \frac{\sin(x)}{y} - \frac{\cos(y)}{x}$
----	------------------------------	---

Задание к лабораторной работе

1. Задание функции и построение графиков
2. Решение уравнения вида $ax+b=f(x)$
3. Функция двух переменных
4. Добавление подписей
5. Оформление данных

Скрипт MATLAB с комментариями

```
clc;
clear;

%% 1. Задание функции и построение графиков
x = -2:0.01:2;
f = @(x) abs(1 - exp(2*x));
y = f(x);

% Символьная производная
syms xs
fs = abs(1 - exp(2*xs));
dfs = diff(fs);
df_num = matlabFunction(dfs); % численная форма
y_derivative = df_num(x);

% Интеграл F(x) = ∫₀ˣ f(y) dy
F = zeros(size(x));
for i = 1:length(x)
    F(i) = integral(f, 0, x(i));
end

% Графики
figure;
subplot(3,1,1);
plot(x, y, 'b', 'LineWidth', 1.5);
title('Функция f(x) = |1 - e^{2x}|');
xlabel('x'); ylabel('f(x)');
legend('f(x) = |1 - e^{2x}|', 'Location', 'northwest');
grid on;

subplot(3,1,2);
plot(x, y_derivative, 'r', 'LineWidth', 1.5);
title('Производная f''(x)');
xlabel('x'); ylabel('f''(x)');
legend('f''(x)', 'Location', 'northwest');
grid on;

subplot(3,1,3);
plot(x, F, 'g', 'LineWidth', 1.5);
title('Интеграл F(x) = ∫₀ˣ f(y)dy');
xlabel('x'); ylabel('F(x)');
legend('F(x) = ∫₀ˣ f(y) dy', 'Location', 'northwest');
grid on;

%% 2. Решение уравнения: a*x + b = f(x)
a = 2;
```

```
grid on;
```

```
%% 2. Решение уравнения:  $a \cdot x + b = f(x)$ 
```

```
a = 2;
```

```
b = 1;
```

```
eq_func = @(x) a*x + b - f(x);
```

```
x0 = -0.3; % начальное приближение
```

```
x_solution = fsolve(eq_func, x0);
```

```
fprintf('Решение уравнения  $2x + 1 = f(x)$ :  $x = %.4f$ \n', x_solution);
```

```
%% 3. Функция двух переменных  $F(x, y) = \sin(x)/y - \cos(y)/x$ 
```

```
[X, Y] = meshgrid(-5:0.25:5, -5:0.25:5);
```

```
% Чтобы избежать деления на ноль:
```

```
X(X == 0) = eps;
```

```
Y(Y == 0) = eps;
```

```
Fxy = sin(X)./Y - cos(Y)./X;
```

```
% Построение 3D-графика
```

```
figure;
```

```
surf(X, Y, Fxy);
```

```
title('F(x, y) = sin(x)/y - cos(y)/x');
```

```
xlabel('x'); ylabel('y'); zlabel('F(x, y)');
```

```
legend('F(x, y)', 'Location', 'best');
```

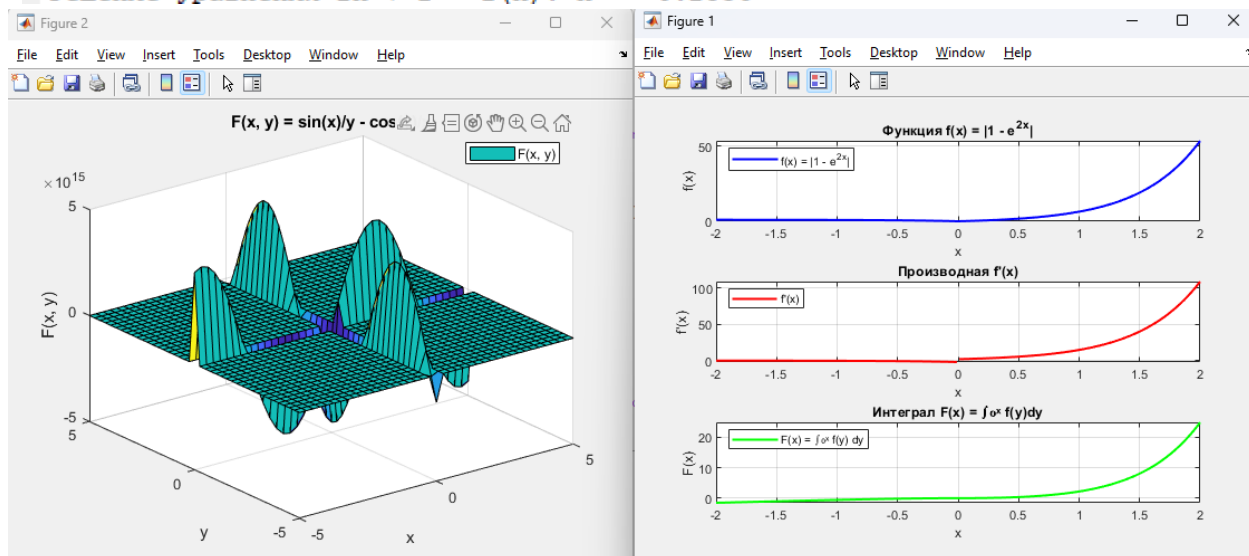
Результаты выполнения

Equation solved.

fsolve completed because the vector of function values is near zero as measured by the value of the function tolerance, and the problem appears regular as measured by the gradient.

<stopping criteria details>

Решение уравнения $2x + 1 = f(x)$: $x = -0.2836$



Контрольные вопросы:

1. Назначение основных окон MATLAB:

Command Window – Основное окно для вывода команд, выполнения операций, вызова функций и отображения результатов

Workspace - Отображает все переменные, созданные в текущем сеансе, вместе с их значениями, типами, размерами и другими свойствами

Current Folder - Показывает содержимое текущей рабочей директории

Figure Window - Отображает графики, построенные с помощью команд типа plot

2. Способы задания переменных в MATLAB:

Прямое присваивание значений: $x = 5$, $A = [1, 2, 3; 1, 2, 3]$ $x = \sqrt{b^2}$

Создание функций: $f = @(x) x.^2 + 5$

Ввод с клавиатуры: $x = \text{input}(\text{'введите переменную икс'})$

3. Использование встроенных функций для вычисления производной и интеграла.

Производная: diff

diff(f) первая производная по переменной по умолчанию

diff(f, x) производная по переменной x

diff(f, x, n) n-я производная по x

$dy = \text{diff}(y) ./ \text{diff}(x)$; производная как $\Delta y / \Delta x$

$x_mid = (x(1:\text{end}-1) + x(2:\text{end})) / 2$; середины интервалов

Интеграл: integral

$f = @(x) \exp(-x.^2)$;

$I = \text{integral}(f, 0, 1)$;

4. Построение двумерных и трёхмерных графиков.

plot – двумерный график

plot(x, y)

title('График функции $y = \sin(x)$ ');

xlabel('x');

ylabel('sin(x)');

grid on;

hold on hold off для нескольких графиков на одном окне

surf – поверхностный график

figure;

surf(X, Y, Z);

title('F(x,y) = $(x^2 + y^2) / \sqrt{x + y}$ ');

xlabel('x');

ylabel('y');

zlabel('F(x,y)');

colorbar;

grid on;

shading interp; (фейд цветов)

Использование функции fsolve для решения уравнений.

Fsolve используется для численного решения систем нелинейных уравнений $F(x) = 0$, где f вектор функция, а x вектор неизвестных

$x = \text{fsolve}(\text{fun}, x0)$

`x = fsolve(fun, x0, options)`

`[x, fval, exitflag, output] = fsolve(fun, x0, options)`

X0 – начальное приближение, опции параметры

алгоритма(опционально), x – найденное решение, fval – значение функции в точке x (должно быть близким к нулю), exitflag – флаг сходимости (положительный – успешное решение), output – информация о процессе решения (итерации, время и т.д.)

5. Форматирование графиков и добавление подписей.

Заголовок – title

Подпись по оси x – xlabel

Подпись по оси Y – ylabel

Подпись по оси Z – zlabel

Легенда – legend (пояснения кривым)

Grid on/off – сетка

Axis – диапазон осей

Лабораторная работа №1.2

Введение в MATLAB. Матричные операции, программирование функций

№	17
варианта	
I	31
J	14

Цель работы: Изучение матричных операций и основ программирования в системе MATLAB.

Задание к лабораторной работе

Создать новый скрипт MATLAB.

Разработать функцию, выполняющую следующие действия:

о Сгенерировать вектор-столбец, элементы которого равномерно распределены в заданном диапазоне.

о Сгенерировать вектор-строку, элементы которого равномерно распределены в заданном диапазоне.

о Перемножить два вектора.

о Вычислить среднее значение и дисперсию матрицы, полученной в результате перемножения векторов.

о В качестве аргумента функции передавать размерности векторов.

3. Произвести расчеты с помощью функции и вывести результаты.

4. Сохранить результаты работы в виде отчета

Скрипт MATLAB с комментариями

```
n = 31; % размер вектора-столбца
m = 14; % размер вектора-строки

% Вызов функции
[meanVal, varVal] = func2(n, m);

% Вывод результатов
fprintf('Среднее значение матрицы: %.4f\n', meanVal);
fprintf('Дисперсия матрицы: %.4f\n', varVal);

function[meanVal, varVal] = myfunc(n,m)
    vc = 10 * rand(n,1); % вектор столбец
    vs = 10 * rand(1,m); % вектор строка
    matrix = vc * vs; % перемножение векторов, для получения матрицы
    meanVal = mean(matrix, 'all'); % считает среднее
    varVal = var(matrix, 0, "all"); % считает дисперсию
end
```

Результаты выполнения

```
>> laba2
Среднее значение матрицы: 21.5355
Дисперсия матрицы: 398.0233
```

Контрольные вопросы:

1. Как определить скалярную переменную в MATLAB?
Скалярная переменная — это переменная, содержащая одно число
MATLAB автоматически определяет тип
2. Как задать вектор-строку и вектор-столбец?
Вектор строка `v_row = [1, 2, 3, 4];` (или через пробел)
Вектор столбец `v_col = [1; 2; 3; 4];`
`v_col = v_row';` - транспонирование
3. Какие функции используются для генерации случайных чисел в MATLAB?
`rand(n,m)` - Матрица случайных чисел от 0 до 1 (равномерное распределение)
`randn(n,m)` - Случайные числа по нормальному распределению (среднее 0, дисперсия 1)
`randi(imax, n, m)` - Случайные числа по нормальному распределению (среднее 0, дисперсия 1)
`randi([a, b], n, m)` - Целые числа от a до b
4. Как выполняется умножение матриц в MATLAB?
Матричное умножение $C = A * B$;

Поэлементное умножение $C = A .* B$;

5. Как вычислить среднее значение и дисперсию в MATLAB?

Среднее значение - $avg = \text{mean}(X)$; среднее по столбцам

$avg_all = \text{mean}(X, 'all')$; среднее по всем элементам

$avg_row = \text{mean}(X, 2)$; среднее по строкам

Дисперсия - $v = \text{var}(X)$; дисперсия по столбцам (по умолчанию деление на $N-1$)

$v0 = \text{var}(X, 0)$; несмещённая ($n-1$) — по умолчанию

$v1 = \text{var}(X, 1)$; смещённая (деление на N)

Лабораторная работа №1.3

Марковские цепи. Определение и построение

Цель работы: Построить матрицу переходов для стохастической маршрутизации в сетях с коммутацией пакетов. Проверить матрицу на стохастичность и цепь Маркова на эргодичность.

Задание к лабораторной работе

1. Построить сеть из $L=15$ узлов в виде ориентированного графа:

- о Каждый узел должен иметь минимум 3 исходящих маршрута.

- о Каждый узел должен иметь хотя бы один входящий маршрут.

2. Построить матрицу переходов $T=\|p_{ij}\|$, задав:

- о Размерность $L=15$

- о Ненулевые элементы вероятностей вручную, чтобы строки удовлетворяли условиям стохастичности.

3. Написать функции в MATLAB:

- о $\text{stochastic}(\text{matrix})$: проверяет матрицу на стохастичность.

- о $\text{ergodic}(\text{matrix}, \text{epsilon})$: проверяет цепь Маркова на эргодичность.

4. Проверить:

- о Стохастичность построенной матрицы.

- о Эргодичность цепи Маркова.

5. Оформить полученные результаты, включая матрицу переходов, функции и результаты проверок

Скрипт MATLAB с комментариями

```
clear; clc;
```

```
%% 1. Построение матрицы переходов (15x15)
```

```
L = 15;
```

```
T = zeros(L);
```

```
% - Каждый узел имеет минимум 3 исходящих маршрута
```

```
% - Каждый узел имеет хотя бы один входящий
```

```
% - Сумма каждой строки = 1 (стохастичность)
```

```
T(1, [2 3 4 5]) = [0.4, 0.2, 0.2, 0.2];
```

```
T(2, [3 6 7 8]) = [0.3, 0.3, 0.2, 0.2];
```

```
T(3, [4 9 10]) = [0.4, 0.4, 0.2];
```

```
T(4, [5 11 12]) = [0.3, 0.5, 0.2];
```

```
T(5, [6 13 14]) = [0.4, 0.3, 0.3];
```

```
T(6, [7 8 9 15]) = [0.3, 0.3, 0.2, 0.2];
```

```
T(7, [8 10 11]) = [0.4, 0.4, 0.2];
```

```
T(8, [9 12 13]) = [0.3, 0.4, 0.3];
```

```
T(9, [10 14 15]) = [0.3, 0.4, 0.3];
```

```
T(10, [11 12 13]) = [0.3, 0.4, 0.3];
```

```
T(11, [12 14]) = [0.5, 0.5];
```

```
T(12, [13 14 15]) = [0.4, 0.3, 0.3];
```

```
T(13, [1 2 3]) = [0.4, 0.3, 0.3];
```

```
T(14, [1 4 5 6]) = [0.3, 0.3, 0.2, 0.2];
```

```
T(15, [1 7 8 9]) = [0.3, 0.4, 0.2, 0.1];
```

```
% Проверка: все строки в сумме дают 1?
```

```
for i = 1:L
```

```
    if abs(sum(T(i,:)) - 1) > 1e-10
```

```
        error('Строка %d не стохастична!', i);
```

```
    end
```

```
end
```

```
%% 2. Проверка на стохастичность
```

```
if stochastic(T)
```

```
    disp('Матрица переходов T - стохастическая.');
```

```
else
```

```
    disp('Матрица переходов T - НЕ стохастическая.');
```

```
end
```

```
%% 3. Проверка на эргодичность
```

```
epsilon = 1e-5;
```

```
if ergodic(T, epsilon)
```

```
    disp('Цепь Маркова - эргодическая.');
```

```
else
```

```
%% 3. Проверка на эргодичность
epsilon = 1e-5;
if ergodic(T, epsilon)
    disp('Цепь Маркова - эргодическая.');
```

```
else
    disp('Цепь Маркова - НЕ эргодическая.');
```

```
end
```

```
%% 4. Вывод матрицы
fprintf('\nМатрица переходов T (15x15):\n');
disp(T);
```

```
%% Функции
```

```
function result = stochastic(matrix)
% Проверяет матрицу на стохастичность

[m, n] = size(matrix);
result = true;

for i = 1:m
    row_sum = sum(matrix(i,:));
    if abs(row_sum - 1) > 1e-10
        result = false; break;
    end
    for j = 1:n
        if matrix(i,j) < 0 || matrix(i,j) > 1
            result = false; break;
        end
    end
    if ~result, break; end
end
end
```

```
function result = ergodic(matrix, epsilon)
% Проверяет цепь Маркова на эргодичность

L = size(matrix, 1);
m = 200;
Tm = matrix^m; % матрица переходов за m шагов

% Тест на разных начальных распределениях
initial_vectors = eye(L); % e1=(1,0...), e2=(0,1,0...) и т.д.

result = true;
```



```

function result = ergodic(matrix, epsilon)
% Проверяет цепь Маркова на эргодичность
|
L = size(matrix, 1);
m = 200;
Tm = matrix^m; % матрица переходов за m шагов

% Тест на разных начальных распределениях
initial_vectors = eye(L); % e1=(1,0...), e2=(0,1,0...) и т.д.

result = true;
for k = 1:L
    pi0 = initial_vectors(k, :); % начальное распределение
    pi_m = pi0 * Tm; % распределение через m шагов

    % Если есть вероятности <= epsilon -> не эргодическая
    if any(pi_m <= epsilon)
        result = false;
        break;
    end
end
end

```

Результаты выполнения

```

Матрица переходов T - стохастическая.
Цепь Маркова - эргодическая.

```

Контрольные вопросы:

1. Определение цепи Маркова.
у процесса нет "памяти" — он помнит только, где находится сейчас, но не помнит, откуда пришёл
2. Классификация цепей Маркова.
Цепи Маркова классифицируются по следующим признакам:

По времени:

С дискретным временем — состояние меняется шаг за шагом (например, каждый день).

С непрерывным временем — переходы могут происходить в любой момент.

По числу состояний:

Конечное число состояний (например, 3: "спит", "идёт", "бежит").

Бесконечное (но счётное) число состояний.

По поведению переходов:

Однородные — вероятности переходов не меняются со временем.

Неоднородные — вероятности зависят от времени.

По структуре связей между состояниями:

Неприводимые, апериодические, эргодические и другие (см. ниже).

3. Свойства цепей Маркова.

Основные свойства:

- Отсутствие памяти: будущее зависит только от настоящего.
- Вероятностные переходы: каждый переход между состояниями происходит с определённой вероятностью.
- Стационарность (для однородных цепей): правила переходов не меняются со временем.
- Сходимость к стационарному состоянию: при определённых условиях со временем распределение вероятностей по состояниям перестаёт меняться.
- Повторяемость: некоторые состояния система посещает снова и снова, другие — только раз.

4. Состояния цепи Маркова.

Состояния могут быть:

- Достижимыми — можно попасть из одного состояния в другое.
- Связанными — можно переходить туда и обратно.
- Поглощающими — попав туда, уже нельзя выйти (как "выпуск" или "отчисление").
- Возвращаемыми — система с высокой вероятностью вернётся в это состояние.
- Временными — возможны, но система в них надолго не задерживается.

5. Что такое апериодическая цепь Маркова?

Апериодическая цепь — это цепь, в которой система может возвращаться в состояние в разные моменты времени без жёсткого периода.

6. Что такое неприводимая цепь Маркова?

Неприводимая цепь — это такая, в которой из любого состояния можно попасть в любое другое состояние (прямо или через промежуточные).

7. Что такое однородная цепь Маркова?

Однородная цепь — это цепь, в которой вероятности переходов между состояниями остаются одинаковыми во времени.

8. Свойство эргодичности.

Эргодичность — это свойство цепи Маркова, при котором:

- Система рано или поздно "забывает" своё начальное состояние.
- Распределение вероятностей по состояниям стабилизируется.
- Устанавливается устойчивое поведение, которое не зависит от того, с чего начался процесс.

Эргодичная цепь — это обычно неприводимая, апериодическая и устойчиво возвращаемая цепь.

9. Стационарное распределение состояний цепи Маркова.

Стационарное распределение — это такое распределение вероятностей по состояниям, которое не меняется со временем.

Если система достигла этого распределения, она будет оставаться в нём дальше.

Оно показывает, какую долю времени система проводит в каждом состоянии в долгосрочной перспективе.

10. Общие понятия о вычислительных сетях с коммутацией пакетов, и методах маршрутизации в них

Сеть с коммутацией пакетов — это тип компьютерной сети, в которой данные разбиваются на небольшие части — пакеты, и каждый пакет передаётся по сети отдельно

Каждый пакет содержит:

- Адрес получателя,
- Номер по порядку,
- Само сообщение (часть файла, голоса и т.д.).

Пакеты могут идти разными путями, а в пункте назначения собираются обратно.

Методы маршрутизации — как пакеты выбирают путь:

- Статическая маршрутизация: пути заданы заранее и не меняются. Просто, но не гибко.
- Динамическая маршрутизация: пути выбираются в зависимости от загрузки сети, обрывов и т.д.
 - Например, по алгоритму "наименьшее время" или "наименьшая загрузка".
- Маршрутизация по таблицам: каждый узел (маршрутизатор) знает, в каком направлении отправлять пакет в зависимости от адреса.
- Адаптивная маршрутизация: система сама подстраивается под изменения в сети.

Такие сети эффективны, устойчивы к сбоям и широко используются в современных технологиях.